

AI ASSISTANT CODING

ASSIGNMENT - 11.3

Name: Saketh Birru

Ht.no: 2303A51403

Batch: 06

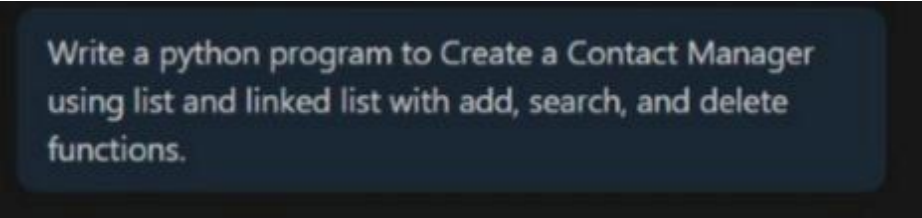
Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

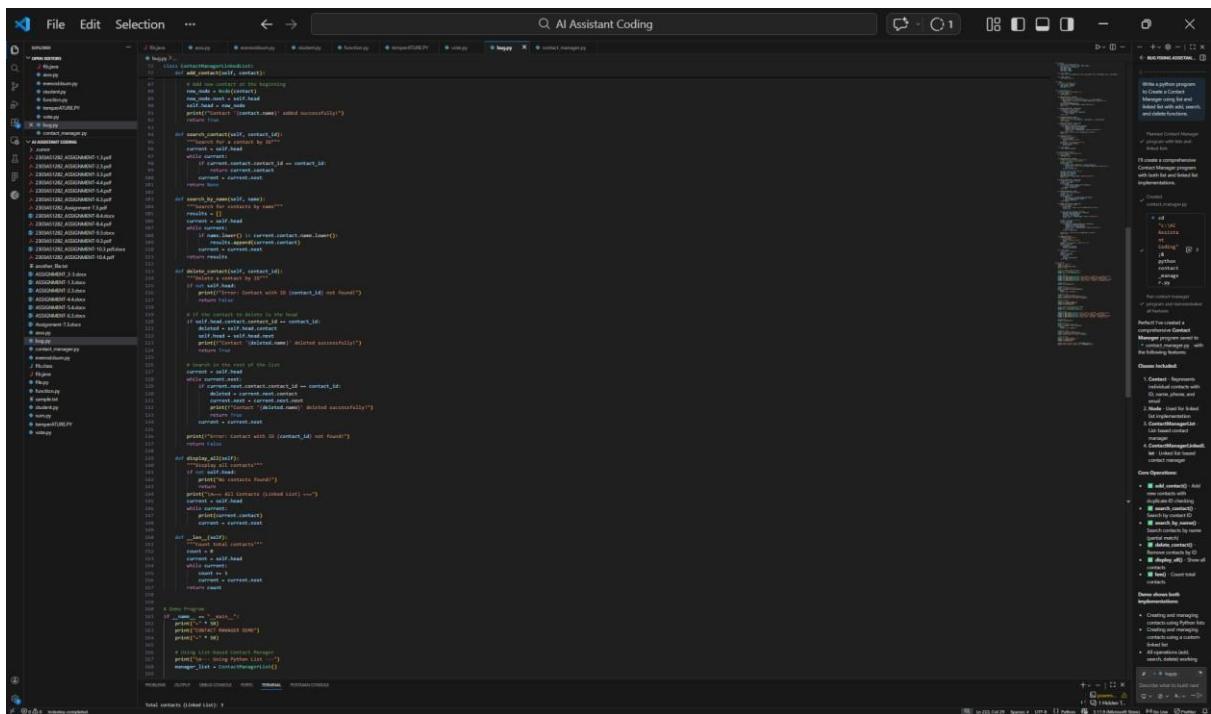
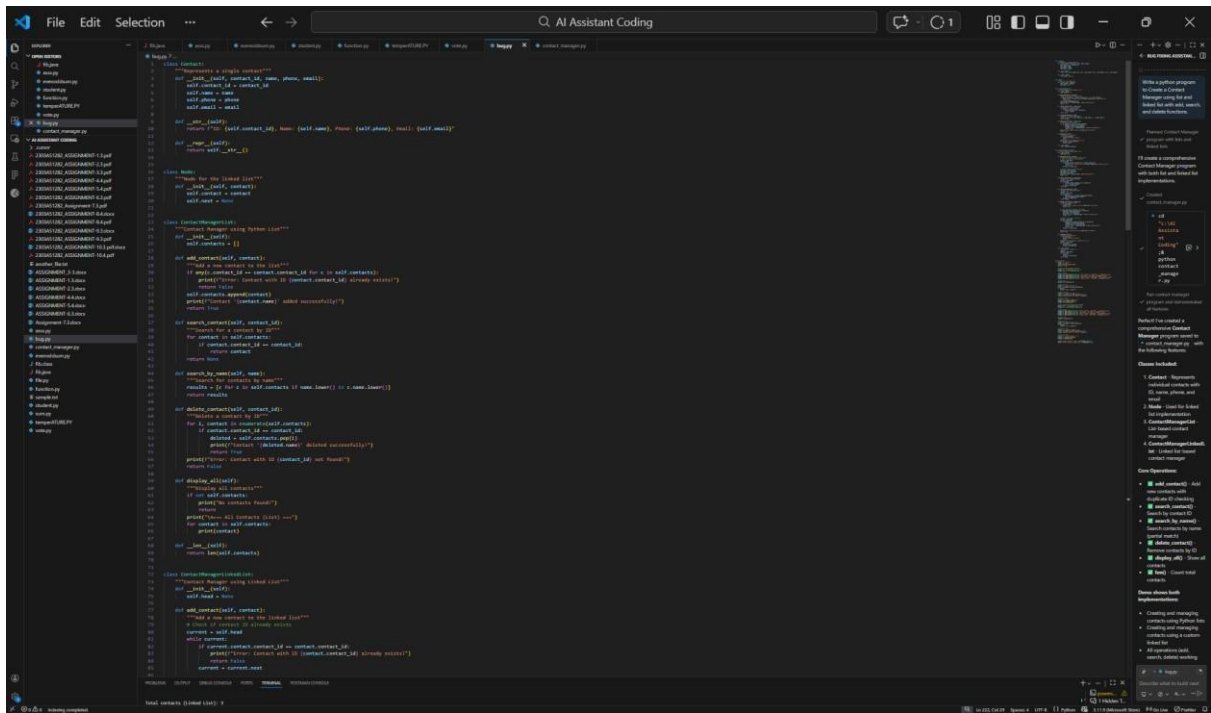
Prompt:

Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.



Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.

Code:





Explanation:

- In an array, adding at the end is fast, but inserting in the middle is slow because elements must shift.
- In a linked list, insertion is fast because no shifting is needed.
- Searching takes the same time in both (you must check each element).
- Deleting in an array is slower due to shifting elements.
- Linked list is better for frequent insertions and deletions.

Task 2: Library Book Search System (Queues & Priority Queues)**Scenario**

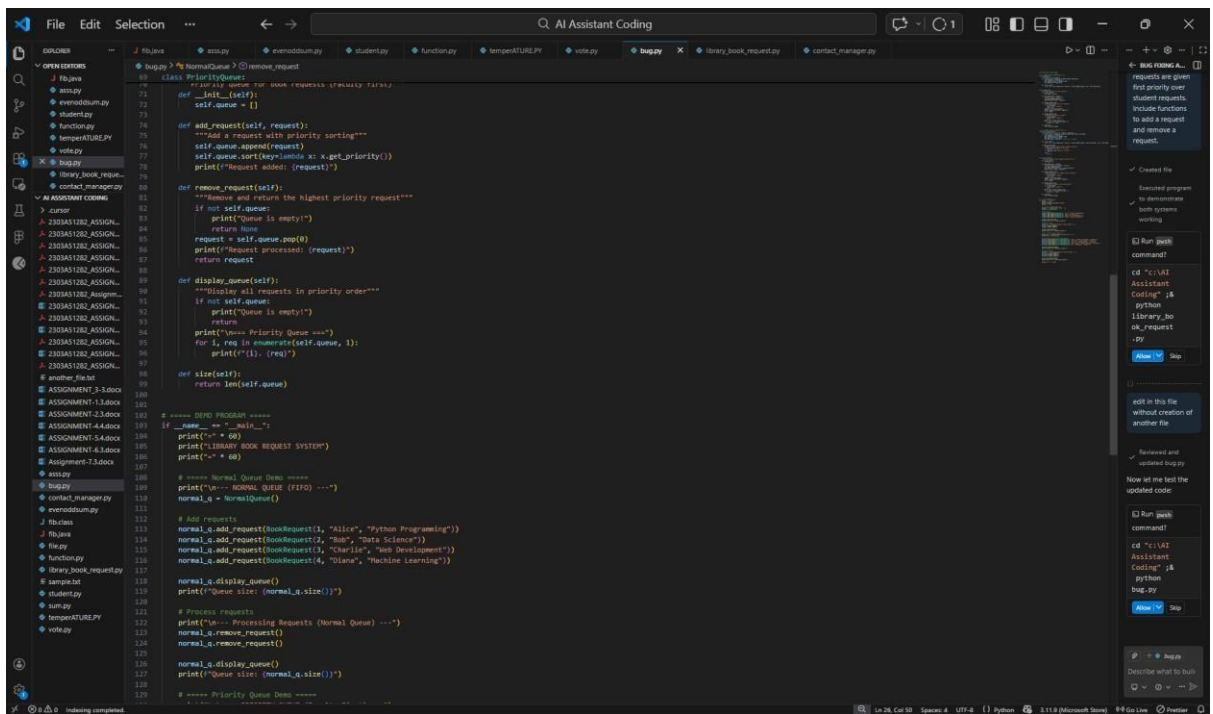
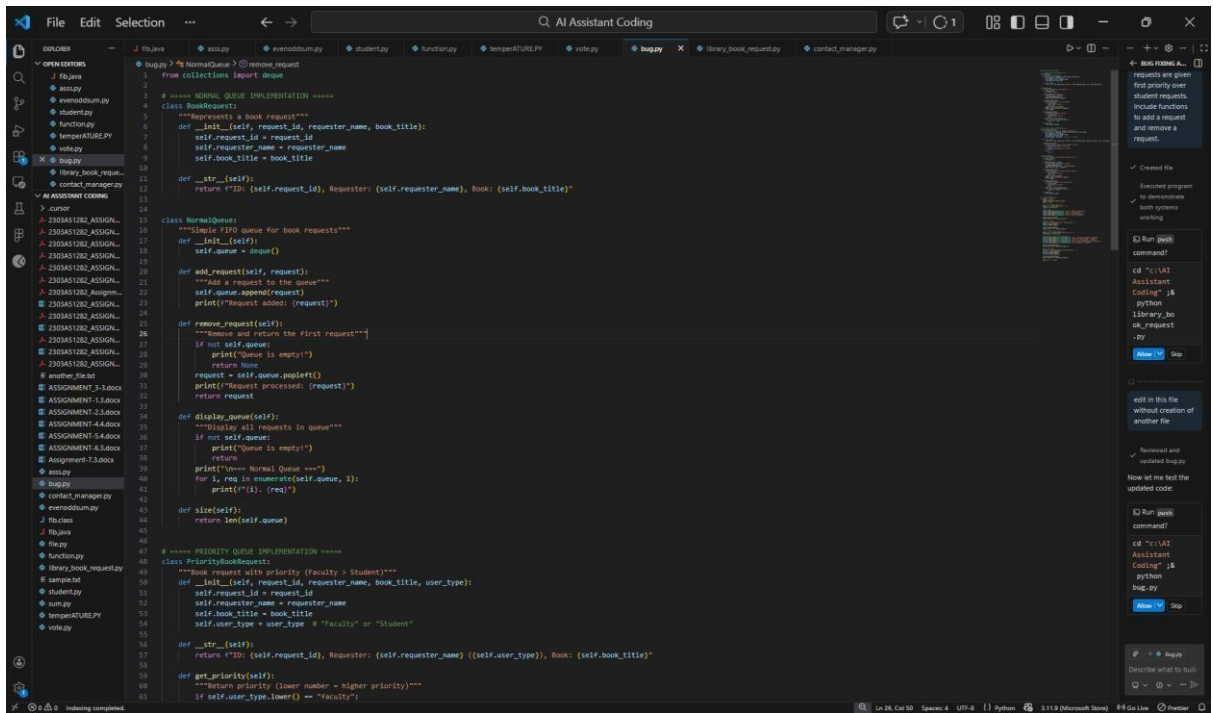
The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

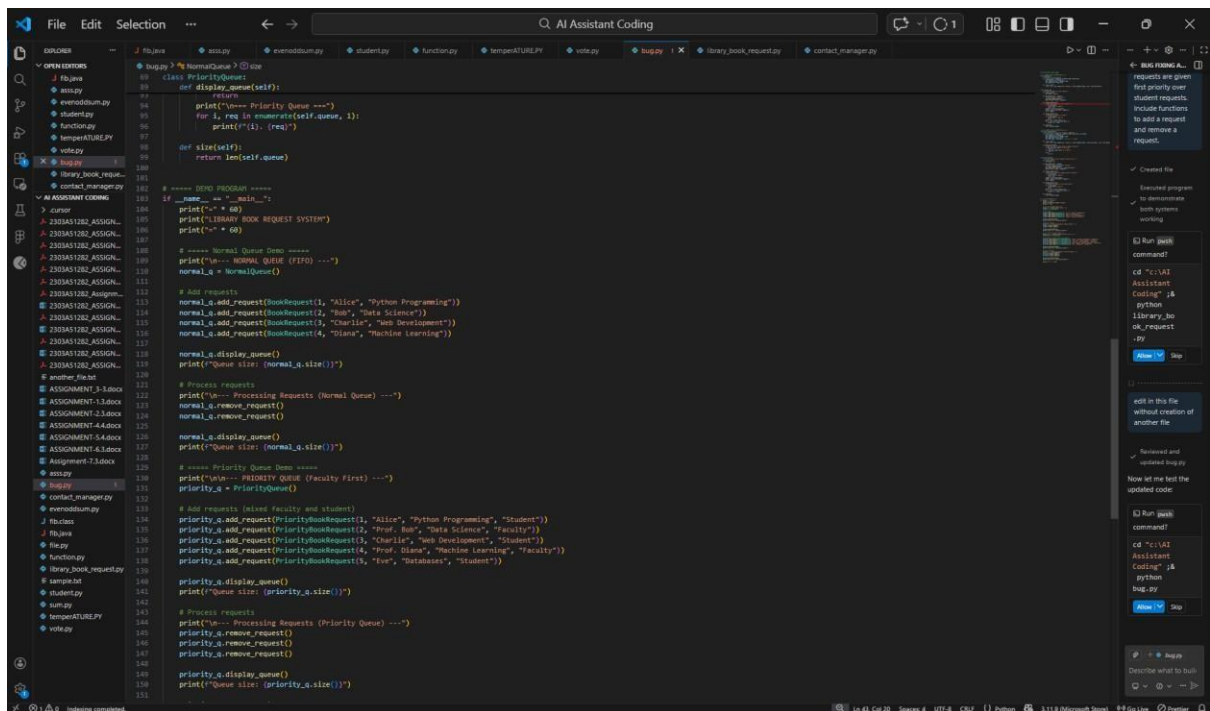
Prompt:

Write a Python program for a library book request system. First, make a normal queue where requests are handled in the order they come. Then, make another version where faculty requests are given first priority over student requests. Include functions to add a request and remove a request.

Write a Python program for a library book request system.
First, make a normal queue where requests are handled in the order they come.
Then, make another version where faculty requests are given first priority over student requests.
Include functions to add a request and remove a request.

Code:





Output:

```

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"

=== Priority Queue ===
1. ID: 3, Requester: Charlie (Student), Book: Web Development
2. ID: 5, Requester: Eve (Student), Book: Databases
Queue size: 2

=====
PS C:\AI Assistant Coding>

```

Explanation:

- Queue (FIFO) → First request comes, first served.(If a student requests first, they get the book first.)

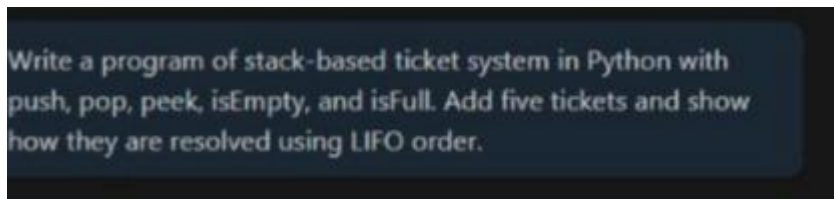
- Priority Queue → Faculty requests are served before students, even if they come later.
- enqueue() → Adds a request to the system.
- dequeue() → Removes and processes the next request.

Task 3: Emergency Help Desk (Stack Implementation) Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Prompt:

Write a program of stack-based ticket system in Python with push, pop, peek, isEmpty, and isFull. Add five tickets and show how they are resolved using LIFO order.



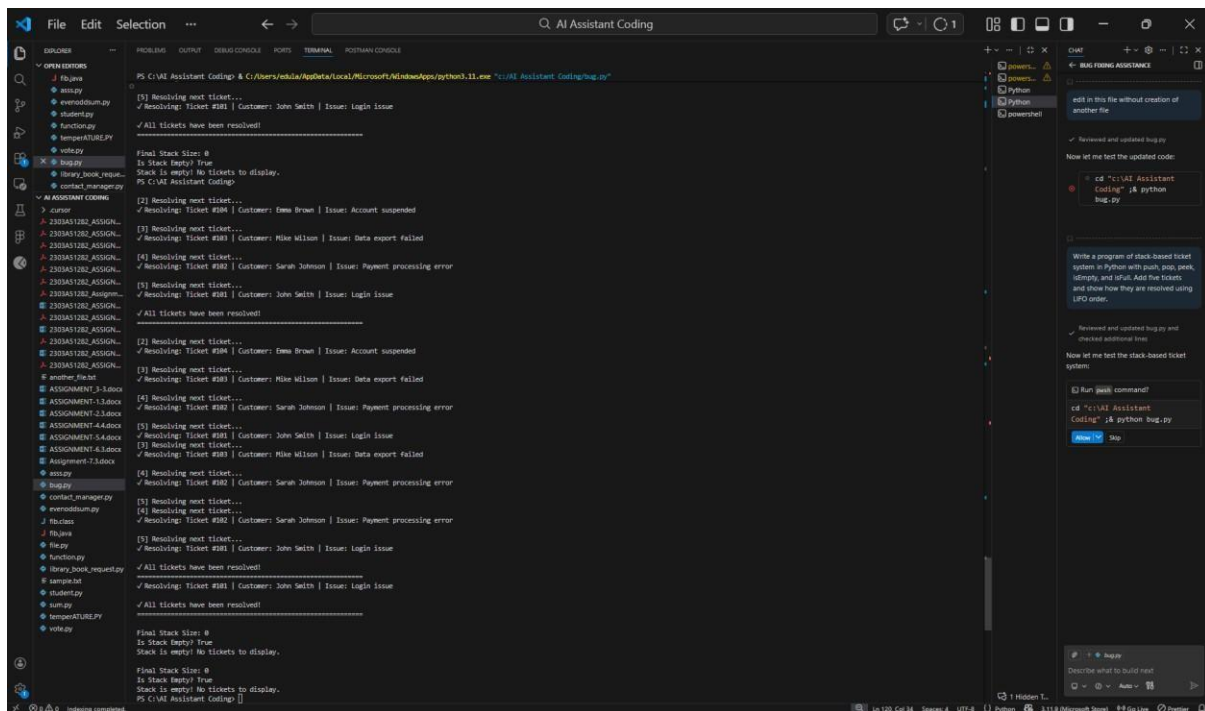
Write a program of stack-based ticket system in Python with push, pop, peek, isEmpty, and isFull. Add five tickets and show how they are resolved using LIFO order.

Code:


```
1 # --- STACK-BASED TICKET SYSTEM ---
2 class TicketStack:
3     """Represents a support ticket"""
4     def __init__(self, ticket_id, customer_name, issue):
5         self.ticket_id = ticket_id
6         self.customer_name = customer_name
7         self.issue = issue
8
9     def __str__(self):
10         return f"Ticket #{self.ticket_id} | Customer: {self.customer_name} | Issue: {self.issue}"
11
12
13 class TicketStack:
14     """Stack-based ticket management system (LIFO - Last In, First Out)"""
15     def __init__(self, max_size=10):
16         self.stack = []
17         self.max_size = max_size
18
19     def push(self, ticket):
20         """Add a ticket to the stack (top of stack)"""
21         if self.is_full():
22             print(f"Error: Stack is full! Cannot add ticket #{ticket.ticket_id}")
23             return False
24         self.stack.append(ticket)
25         print(f"/ Ticket added: {ticket}")
26         return True
27
28     def pop(self):
29         """Remove and return the ticket from the top of the stack"""
30         if self.is_empty():
31             print(f"Error: Stack is empty! No tickets to resolve.")
32             return None
33         ticket = self.stack.pop()
34         print(f"/ Resolving: {ticket}")
35         return ticket
36
37     def peek(self):
38         """View the top ticket without removing it"""
39         if self.is_empty():
40             print(f"Error: Stack is empty!")
41             return None
42         return self.stack[-1]
43
44     def is_empty(self):
45         """Check if the stack is empty"""
46         return len(self.stack) == 0
47
48     def is_full(self):
49         """Check if the stack is full"""
50         return len(self.stack) >= self.max_size
51
52     def size(self):
53         """Return the number of tickets in the stack"""
54         return len(self.stack)
55
56     def display_stack(self):
57         """Display all tickets in the stack (top to bottom)"""
```

```
113 class TicketStack:
114     print(f"<< ==>>")
115
116     def resolve_all(self):
117         """Resolve all tickets in LIFO order"""
118         print(f"<< ==>>")
119         print(f"RESOLVING ALL TICKETS (LIFO - Last In, First Out)")
120         print(f"<< ==>>")
121         count = 1
122         while not self.is_empty():
123             print(f"\n{count}: Resolving next ticket...")
124             self.pop()
125             count += 1
126         print(f"\n All tickets have been resolved!")
127         print(f"<< ==>>")
128
129 # ===== DEMO PROGRAM =====
130 if __name__ == "__main__":
131     print(f"<< ==>>")
132     print(f"STACK-BASED TICKET SYSTEM")
133     print(f"<< ==>>")
134
135     # Create ticket stack with max size of 10
136     ticket_system = TicketStack(max_size=10)
137
138     # Add five tickets
139     print(f"\n--- ADDING TICKETS TO THE STACK ---\n")
140     ticket_system.push(ticket(101, "John Smith", "Single issue"))
141     ticket_system.push(ticket(102, "Sarah Johnson", "Payment processing error"))
142     ticket_system.push(ticket(103, "Mike Wilson", "Data export failed"))
143     ticket_system.push(ticket(104, "Tina Brown", "Account suspension"))
144     ticket_system.push(ticket(105, "David Lee", "Password reset not working"))
145
146     # Display current stack
147     ticket_system.display_stack()
148
149     # Show stack information
150     print(f"\nStack Size: {ticket_system.size()}")
151     print(f"Is Stack Empty? {ticket_system.is_empty()}")
152     print(f"Is Stack Full? {ticket_system.is_full()}")
153
154     # Peek at the top ticket
155     print(f"\n--- PEER AT TOP TICKET ---")
156     top_ticket = ticket_system.peek()
157     if top_ticket:
158         print(f"Top ticket (without removing): {top_ticket}")
159
160     # Resolve all tickets in LIFO order
161     ticket_system.resolve_all()
162
163     # Display final stack state
164     print(f"\nFinal Stack Size: {ticket_system.size()}")
165     print(f"Is Stack Empty? {ticket_system.is_empty()}")
166     ticket_system.display_stack()
```

Output:



Explanation:

The program uses a stack to manage help desk tickets.

A stack works in last in, first solved order.

When a new ticket is raised, it is added to the top.

When solving a ticket, the most recent one is handled first.

The program can also check if there are no tickets left or if the stack is full.

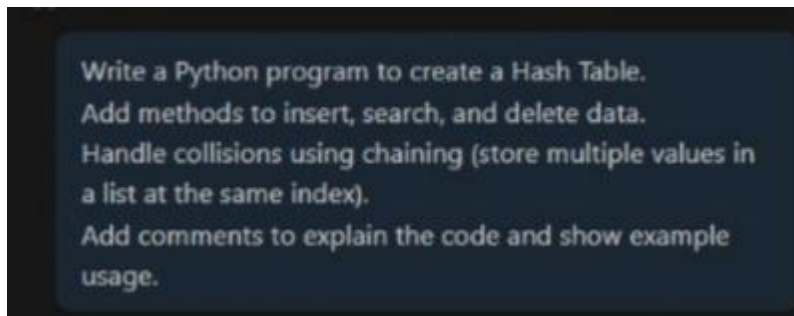
Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Prompt:

Write a Python program to create a Hash Table.
Add methods to insert, search, and delete data. Handle collisions using chaining (store multiple values in a list at the same index).
Add comments to explain the code and show example usage.



Code:

```
File Edit Selection ... Q AI Assistant Coding
# Create Hash Table with CHAINING collision handling using
# key-value pair (list for storing data)
# key-value pair (list for storing data)
class HashTable:
    """Represents a key-value pair in the hash table"""
    def __init__(self, key, value):
        self.key = key
        self.value = value
    def __str__(self):
        return f'({self.key}: {self.value})'

# Create Hash Table
class HashTable:
    """Hash Table Implementation using chaining to handle collisions.
    Chaining stores multiple key-value pairs that hash to the same index in a linked list.
    """
    def __init__(self, size=10):
        """Initialize the hash table
        Args:
            size (int): Number of buckets in the hash table
        """
        self.size = size
        # Create an array of empty lists (chains) for collision handling
        self.table = [[] for _ in range(size)]
        self.total_items = 0

    def _hash_function(self, key):
        """Hash function to determine the index for a given key
        Uses simple modulo operation: hash(key) % table_size
        Args:
            key: The key to hash
        Returns:
            int: Index in the hash table
        """
        # Convert key to string and sum ASCII values for different key types
        key_str = str(key)
        hash_value = sum(ord(char) for char in key_str)
        return hash_value % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table
        If key already exists, update its value
        If collision occurs, add to the chain (list) at that index
        Args:
            key: The key to insert
            value: The value associated with the key
        Returns:
            bool: True if insertion successful, False otherwise
        """
        # Get the hash index
        index = self._hash_function(key)
        chain = self.table[index]

        # Check if key already exists in the chain and update if found
        for i, pair in enumerate(chain):
            if pair.key == key:
                chain[i] = (key, value)
                return True

        # Add new key-value pair to the chain
        chain.append((key, value))
        self.total_items += 1
        return True

    def search(self, key):
        """Search for a key in the hash table
        Args:
            key: The key to search for
        Returns:
            list: List of values associated with the key, or empty list if not found
        """
        index = self._hash_function(key)
        chain = self.table[index]

        for pair in chain:
            if pair.key == key:
                return [pair.value]

        return []

    def delete(self, key):
        """Delete a key-value pair from the hash table
        Args:
            key: The key to delete
        Returns:
            bool: True if deletion successful, False otherwise
        """
        index = self._hash_function(key)
        chain = self.table[index]

        for i, pair in enumerate(chain):
            if pair.key == key:
                chain.pop(i)
                self.total_items -= 1
                return True

        return False

# Example Usage
if __name__ == '__main__':
    # Create a Hash Table
    ht = HashTable()

    # Insert key-value pairs
    ht.insert('apple', 1)
    ht.insert('banana', 2)
    ht.insert('apple', 3)  # This will update the value of 'apple' to 3
    ht.insert('orange', 4)
    ht.insert('banana', 5)  # This will add to the chain for 'banana'

    # Search for a key
    search_results = ht.search('apple')
    print(f"Search results for 'apple': {search_results}")

    # Delete a key-value pair
    ht.delete('orange')

    # Search for a key after deletion
    search_results = ht.search('orange')
    print(f"Search results for 'orange': {search_results}")

    # Print the total number of items
    print(f"Total items in the hash table: {ht.total_items}")
```

```
def insert(self, key, value):
    """Insert a key-value pair into the hash table. If found, update the value.
    If not found, add a new key-value pair to the chain.
    """
    # Check if key already exists in the chain and update if found
    for i, pair in enumerate(self.chain):
        if pair.key == key:
            self.chain[i].value = value
            print(f"Updated: {key} = {value} (at index {i})")
            return True

    # If key not found, add new key-value pair to the chain
    self.chain.append((key, value))
    print(f"Inserted: {key} = {value} (at index {len(self.chain)-1})")
    return True

def search(self, key):
    """Search for a key in the hash table.
    Args:
        key: The key to search for.
    Returns:
        The value associated with the key, or None if not found.
    """
    # Search for the key in the chain
    for i, pair in enumerate(self.chain):
        if pair.key == key:
            print(f"Found: {key} = {pair.value} (at index {i})")
            return pair.value

    # Key not found
    print(f"Key '{key}' not found in the hash table")
    return None

def delete(self, key):
    """Delete a key-value pair from the hash table.
    Args:
        key: The key to delete.
    Returns:
        True if deletion successful, False if key not found.
    """
    # Search for the key in the chain
    for i, pair in enumerate(self.chain):
        if pair.key == key:
            del self.chain[i]
            print(f"Deleted: {key} = {pair.value} (from index {i})")
            return True

    # Key not found
    print(f"Key '{key}' not found in the hash table")
    return False

def display(self):
    """Display all key-value pairs in the hash table.
    """
    for i, pair in enumerate(self.chain):
        print(f"{i}: {pair.key} = {pair.value}")
```

```
def get_all_items(self):
    """Return all key-value pairs in the hash table.
    """
    all_items = []
    for pair in self.chain:
        all_items.append(pair)
    return all_items

# ===== INITIALIZATION =====
def __init__(self):
    """Initialize the hash table with 10 buckets.
    """
    self.buckets = [[] for _ in range(10)]
    self.chain = []

# ===== INSERT OPERATIONS =====
def insert(self, key, value):
    """Insert a key-value pair into the hash table.
    """
    bucket_index = hash(key) % len(self.buckets)
    self.buckets[bucket_index].append((key, value))

def search(self, key):
    """Search for a key in the hash table.
    """
    bucket_index = hash(key) % len(self.buckets)
    for pair in self.buckets[bucket_index]:
        if pair.key == key:
            return pair.value
    return None

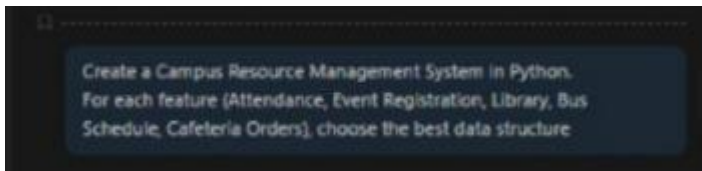
def delete(self, key):
    """Delete a key-value pair from the hash table.
    """
    bucket_index = hash(key) % len(self.buckets)
    for i, pair in enumerate(self.buckets[bucket_index]):
        if pair.key == key:
            del self.buckets[bucket_index][i]
            return True
    return False

def display(self):
    """Display all key-value pairs in the hash table.
    """
    for bucket_index, bucket in enumerate(self.buckets):
        for i, pair in enumerate(bucket):
            print(f"{bucket_index}: {i}: {pair.key} = {pair.value}")
```

Output:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue **Prompt:**

Create a Campus Resource Management System in Python. For each feature (Attendance, Event Registration, Library, Bus Schedule, Cafeteria Orders), choose the best data structure



Code:

```

1  # Campus Resource Management System
2  # Features and chosen data structures:
3  # - Attendance: set (O(1) add/remove) to track present student IDs
4  # - Event Registration: deque (FIFO queue) for fair registration processing
5  # - Library: dict (hash table) mapping ISBN -> book record for fast lookup
6  # - Bus Schedule: dict of route -> sorted list of departure times (list kept sorted)
7  # - Cafeteria Orders: heap (priority queue) to prioritize faculty over students while preserving arrival order
8
9  Run this file to see a small demo of each feature.
10
11
12
13  from collections import deque
14  import heapq
15  import heapq
16  from bisect import bisect
17  from datetime import datetime, timedelta
18
19  # ----- Attendance (set) -----
20  class Attendance:
21  """Track attendance using a set for O(1) add/remove/check."""
22  def __init__(self):
23      self.present = set()
24
25  def mark_present(self, student_id):
26      self.present.add(student_id)
27      print(f"Marked present: {student_id}")
28
29  def mark_absent(self, student_id):
30      self.present.discard(student_id)
31      print(f"Marked absent: {student_id}")
32
33  def is_present(self, student_id):
34      return student_id in self.present
35
36  def present_count(self):
37      return len(self.present)
38
39  def list_present(self):
40      return sorted(self.present)
41
42  # ----- Event Registration (FIFO queue) -----
43  class EventRegistration:
44  """Register attendees in arrival order using deque."""
45  def __init__(self):
46      self.queue = deque()
47
48  def add_registration(self, attendee_id, name):
49      self.queue.append((attendee_id, name))
50      print(f"Registered: {attendee_id} - {name}")
51
52  def process_registrations(self):
53      if not self.queue:
54          print("No registrations to process.")
55          return None
56      attendee = self.queue.popleft()
57      print(f"Processed registration: {attendee[0]} - {attendee[1]}")
58      return attendee
59
60  def pending_count(self):
61      return len(self.queue)

```

```
class EventRegistration:
    def __init__(self):
        self.pending_count = 0
        self.pending_list = []

    def add_pending(self):
        return self.pending_list

    def remove_pending(self):
        return self.pending_list

class Library:
    """Library using a dict for O(1) lookups by ISBN."""
    def __init__(self):
        self.lib = {}
        self.catalog = {}

    def add_book(self, isbn, title, author, copies=1):
        if isbn in self.catalog:
            self.catalog[isbn]['copies'] += copies
        else:
            self.catalog[isbn] = {
                'title': title,
                'author': author,
                'copies': copies,
                'borrowers': []
            }
        print(f"Added book: {title} (ISBN: {isbn}).")

    def search(self, isbn):
        return self.catalog.get(isbn)

    def borrow_book(self, isbn, user_id):
        book = self.catalog.get(isbn)
        if not book:
            print("Book not found.")
            return False
        if book['copies'] <= 0:
            print("No copies available.")
            return False
        book['copies'] -= 1
        book['borrowers'].append(user_id)
        print(f"User {user_id} borrowed book: {title}")
        return True

    def return_book(self, isbn, user_id):
        book = self.catalog.get(isbn)
        if not book:
            print("Book not found.")
            return False
        try:
            book['borrowers'].remove(user_id)
        except ValueError:
            print(f"User {user_id} did not borrow the book.")
            return False
        book['copies'] += 1
        print(f"User {user_id} returned book: {title}")
        return True

    def list_available(self):
        return [(isbn, info['title'], info['copies']) for isbn, info in self.catalog.items()]

class BusSchedule:
    """Bus Schedule (Route > Sorted list of times)"""
    def __init__(self):
        self.routes = {}

    def add_route(self, route, times):
        self.routes[route] = sorted(times)

    def get_route(self, route):
        return self.routes.get(route, [])

    def is_route(self, route):
        return route in self.routes

    def get_next(self, route, current_time):
        times = self.routes[route]
        for time in times:
            if time > current_time:
                return time
        return None
```

```
def main():
    # Attendance demo
    att = Attendance()
    att.add_present('5801')
    att.add_present('5802')
    att.add_present('5803')
    print(f"Present list: {att.get_present_list()}")
    print(f"Present count: {att.get_present_count()}")
    att.add_absent('5802')
    print(f"Present count: {att.get_present_count()}")

    # Event registration demo
    ev = EventRegistration()
    ev.add_registration('A001', 'Alice')
    ev.add_registration('A002', 'Bob')
    ev.add_registration('A003', 'Charlie')
    print(f"Pending registrations: {ev.get_pending_list()}")
    ev.process_registration()
    print(f"Pending count: {ev.get_pending_count()}")

    # Library demo
    lib = Library()
    lib.add_book('978-013166307', 'Clean Code', 'Robert C. Martin', copies=2)
    lib.add_book('978-013166308', 'Fluent Python', 'Luciano Ramalho', copies=1)
    lib.borrow_book('978-013166307', '5801')
    lib.borrow_book('978-013166307', '5802')
    lib.borrow_book('978-013166307', '5803')
    lib.borrow_book('978-013166307', '5804')
    print(f"Available books: {lib.get_list_available()}")
    lib.return_book('978-013166307', '5801')
    print(f"Available books after return: {lib.get_list_available()}")

    # Bus schedule demo
    bs = BusSchedule()
    bs.add_route('Route A', [10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
    bs.add_route('Route B', [15, 25, 35, 45, 55, 65, 75, 85, 95, 105])
    bs.add_route('Route C', [20, 30, 40, 50, 60, 70, 80, 90, 100, 110])
    print(f"Next Route A bus: {bs.get_next('Route A', current_time=0)}")
    print(f"Next Route B bus: {bs.get_next('Route B', current_time=0)}")
    print(f"Next Route C bus: {bs.get_next('Route C', current_time=0)}")

    # Cafeteria orders demo
    caf = CafeteriaOrders()
    caf.add_order('0801', '0801', ['Coffee', 'Sandwich'], customer_type='Student')
    caf.add_order('0802', '0801', ['Salad'], customer_type='Faculty')
    caf.add_order('0803', '0802', ['Tea'], customer_type='Student')
    caf.add_order('0804', '0802', ['Pasta'], customer_type='Faculty')
    print(f"Pending cafeteria orders: {caf.get_pending_list()}")
    caf.serve_order()
    print(f"Pending orders after serving: {caf.get_pending_list()}")

    print("Vollum complete.")
```

Output:


```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL POSTMAN CONSOLE

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
Is empty: False
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/campus_resource_management.py"
-----
Campus Resource Management Demo
-----
Marked present: 5001
Marked present: 5002
Marked present: 5010
Present list: ['5001', '5002', '5010']
Is 5002 present? True
Marked absent: 5002
Present count: 2
Registered: A001 - Alice
Registered: A002 - Bob
Registered: A003 - Charlie
Pending registrations: [('A001', 'Alice'), ('A002', 'Bob'), ('A003', 'Charlie')]
Processed registration: A001 - Alice
Pending count: 2
Added book: Clean Code (ISBN: 978-0135166307).
Added book: Fluent Python (ISBN: 978-1491958296).
5001 borrowed Clean Code
5003 borrowed Clean Code
No copies available.
Available books: [('978-0135166307', 'Clean Code', 0), ('978-1491958296', 'Fluent Python', 1)]
5001 returned Clean Code
Available books after return: [('978-0135166307', 'Clean Code', 1), ('978-1491958296', 'Fluent Python', 1)]
Added bus time for Route A: 2026-02-18 10:42:24.367227
Added bus time for Route A: 2026-02-18 10:57:24.367227
Added bus time for Route B: 2026-02-18 10:39:24.367227
Next Route A bus: 2026-02-18 10:42:24.367227
Route A schedule: [datetime.datetime(2026, 2, 18, 10, 42, 24, 367227), datetime.datetime(2026, 2, 18, 10, 57, 24, 367227)]
Order added: 0001 (Student)
Order added: 0002 (Faculty)
Order added: 0003 (Student)
Order added: 0004 (Faculty)
Pending cafeteria orders: 4
Serving order: 0002 (Faculty)
Serving order: 0004 (Faculty)
Pending orders after serving: 2

Demo complete.
PS C:\AI Assistant Coding> 
```

Explanation:

Library Book Borrowing using a queue:

- The queue stores student names who request a book.
- When a student requests a book, we use enqueue() to add them to the queue.
- When a book becomes available, we use dequeue() to give it to the first student in line.
- This ensures fairness because the first requester gets the book first.

